FOR ENGINEERING, CONTROL SYSTEMS AND IT TEAMS

Your readings belong in the screen your operators already watch.

FlowSense holds a validated record of your network: flow, level, rainfall, pump station behaviour and everything derived from them. That record is yours, and it does not have to stay in our browser. There are four ways out, all of them standing feeds rather than exports somebody remembers to run, and between them they reach any SCADA package, historian, reporting tool or work management system we have met. Nothing is installed at your end, and if nothing new may be exposed, one of the four is entirely outbound from you.

FOUR WAYS OUT

Scheduled CSV files onto your server, MQTT messages to your broker, a read API you pull on your own schedule, and signed webhooks the moment an alarm fires.

WHAT YOU INSTALL

Nothing. No agent, no connector, no gateway appliance, at your sites or in your control room.

HOW OFTEN

Anything from every minute to once a month, anchored to a clock time if a nightly file suits your ETL better.

WHAT ARRIVES

Readings as measured, in your column names, your units, your time zone and your delimiter. No smoothing, no gap filling, no interpretation.

GAP FREE BY DESIGN

A delivery window only closes once every file has landed. A failed night re-sends itself, so a duplicate is possible and a missing reading is not.

TIME TO FIRST FILE

About an hour once the destination account exists, including the connection test and a preview of the exact file before anything is sent.

1. THE SHORT VERSION

One record, four ways out, no copy of it left stale.

Every reading in FlowSense sits on a named channel, with its units, its site and its own field timestamp. A feed is a standing selection of those channels with a destination attached. Once it is running, your systems see the same numbers we do, on the cadence you asked for, without anybody exporting anything.

Your network

Meters, rain gauges, pump stations and the loggers or SCADA that already collect them.

THEN
into the record

The FlowSense record

Named channels with units and a time zone, plus derived flow, rainfall and analytics.

ONE OF FOUR
ways out

Your receiver

A folder on your server, a topic on your broker, a script that pulls, or an endpoint that listens.

THEN
your systems

SCADA, historian, BI, work orders

The tags, trends, reports and jobs your operators already use.

Three things worth knowing before you read any further

- **What leaves is what was measured.** Values are sent exactly as stored, with no gap filling and no smoothing. Where a reading is missing the field is blank, never a zero and never a placeholder, so your own quality rules see the gap rather than a plausible number.
- **A re-sent window is normal, a missing reading is not.** A delivery advances its watermark only after every file has landed, so an outage re-sends the same window rather than skipping it. Make the receiving end idempotent on tag and timestamp and this costs you nothing.
- **Nothing new has to be exposed.** If your systems pull from us, or if your endpoint already answers on HTTPS, there is no firewall change at all. If we push to you, it is one service you already run, reachable from a single EDS source address you allowlist.

2. WHICH WAY OUT SUITS WHAT YOU RUN

Pick by what the receiving system reads, not by protocol.

Almost every system on this list can take data three different ways. The one to choose is whichever your team already supports, because that is the one nobody has to learn, maintain or hand over.

IF THIS IS WHAT HAS TO RECEIVE THE DATA	THE WAY OUT	WHO OPENS THE CONNECTION
Anything with an MQTT client: Ignition with MQTT Engine, Node-RED, Mosquitto, EMQX, HiveMQ, and the brokers most integration layers now sit on	MQTT messages published to your broker, JSON or CSV The closest thing to live. Messages arrive as the readings do, on topics you name.	We connect out to you.
SCADA packages that import files or watch a folder: Geo SCADA Expert, Plant SCADA (Citect), VTScada, WinCC OA, AVEVA System Platform	Scheduled CSV files onto your SFTP or FTPS server	We connect out to you.
Historians and time series stores: AVEVA Historian (Wonderware), AVEVA PI System through its file loader, Kisters Hydstra and WISKI, Aquarius, eagle.io	Scheduled CSV files, in the layout and column names the loader expects	We connect out to you.
Reporting, BI and GIS: Power BI, Excel, Grafana, Tableau, ArcGIS, a council data warehouse, a consultant's model	The read API, pulled on your own schedule with an API key	You connect out to us. Nothing inbound.
Anything that should react rather than poll: paging and on-call tools, work order and maintenance systems, an integration bus, a Teams or Slack channel through a connector	Signed webhooks, posted the moment an alarm fires or clears, a forecast puts a site at risk, or Blockage Watch raises a blockage warning	We post to an endpoint you publish.
Nothing at all, because somebody just wants the numbers	CSV from the screen, or a scheduled PDF and CSV report by email No integration, no account on either side, no IT involvement.	Neither. It is a download or an email.

3. THE FOUR WAYS OUT, IN THE DETAIL AN INTEGRATOR NEEDS

Each one answers the same four questions.

Who opens the connection, what you provide, what arrives on the wire, and what happens when something fails. If you are scoping the work, this section is the scope.

WAY OUT ONE

Scheduled CSV files onto your server

We connect to you, over SFTP or FTPS

The workhorse, and the one most historians and SCADA importers are happiest with. You provide an account confined to one folder. We write a file into it on your schedule, in your format, and your existing importer picks it up exactly as it would a file

written by anything else.

What you provide	A host name and port, a folder, and an account that can write to it, with either a password or an SSH key. The account needs nothing else: no shell, no access outside the folder.
What arrives	UTF-8 CSV with CRLF line endings, quoted to RFC 4180, named from a template you set. Section 4 sets out every column.
Never half a file	Each file is written under a temporary name and renamed into place, so a folder watcher can never read one that is still being written. On by default and worth leaving on. This is the single most common cause of corrupt rows in a file-based integration, and the rename removes it rather than making your side defend against it.
Before a credential is sent	We scan your server, show the operator the SSH host key or TLS certificate it actually presented, and pin it. A mismatch after that aborts the connection before the password is offered.
When it fails	The window is not closed, so it is re-sent. Retries back off through 5, 10, 20, 40 and 60 minutes, never slower than the feed's own interval, and every attempt is in the run log with the rows, bytes and remote paths it wrote.

A CONSOLIDATED FILE, FOUR CHANNELS ACROSS THREE STATIONS

```
# Timestamp timezone: Australia/Brisbane
timestamp,"Milton St PS, Flow (L/s)","Milton St PS, Level (m)","Riverbank Trunk, Flow (L/s)","Depot Gauge, Rainfall (mm)"
2026-09-16 02:00:00,12.42,0.184,48.31,0.0
2026-09-16 02:05:00,11.98,0.179,47.66,0.2
2026-09-16 02:10:00,,,47.51,0.2
```

Read left to right: the time zone is stated in the file rather than assumed, the header carries units, and the two blank fields at 02:10 are a station that had not reported yet, not a pair of zeroes. Set a column alias per channel and those headers become your own tag names instead, which is what a historian loader usually wants. Delivered as `Council_SCADA_feed_20260916_021504.csv` from the default name template.

WAY OUT TWO

MQTT messages published to your broker

We connect to you, over MQTT, with or without TLS

If your integration layer already runs a broker, this is the shortest path from our record to your tags. The same readings a file feed would carry are published as messages instead, on topics you name, and whatever subscribes to them gets the data without a file ever existing.

What you provide	A broker address and a topic prefix. A publish-only account if your broker uses authentication, and nothing at all if it does not: anonymous brokers on a closed network are normal and are accepted.
Topics	A template you set, defaulting to `eds/{feed}/{site}/{channel}` . Slashes in the template are your hierarchy; the substituted values are sanitised, so a station named "Griffiths St / North" cannot invent a level or smuggle in a wildcard. On a consolidated feed the site and channel levels resolve to `all` , so a subscriber's topic never moves when one channel goes quiet.
Payload	JSON by default, or the same CSV bytes a file feed would have uploaded, one message per file. Units travel once per channel in the descriptors rather than being baked into every key.
Delivery	QoS 1 by default, because at QoS 0 the broker never acknowledges and a publish the network drops cannot be detected. Retained messages are off unless you ask for them.
TLS	Optional and recommended. Your certificate or internal CA is pinned at setup and host name checking stays on, including when the certificate is self signed.

ONE MESSAGE, SHOWN FORMATTED (IT IS PUBLISHED ON A SINGLE LINE)

```
topic: eds/Council_SCADA_feed/Milton_St_PS/all

{
  "feed": "Council SCADA feed",
  "generated_at": "2026-09-16T02:15:04Z",
  "timezone": "Australia/Brisbane",
  "site": "Milton St PS",
  "channels": [
    {"key": "Flow", "channel_id": 1841, "name": "Flow", "units": "L/s",
     "site_id": 204, "site": "Milton St PS"},
    {"key": "Level", "channel_id": 1842, "name": "Level", "units": "m",
     "site_id": 204, "site": "Milton St PS"}
  ],
  "count": 2,
  "readings": [
    {"timestamp": "2026-09-16 02:05:00", "values": {"Flow": 11.98, "Level": 0.179}},
    {"timestamp": "2026-09-16 02:10:00", "values": {"Flow": 12.05}}
  ]
}
```

A channel with no reading at a timestamp is absent from the map rather than sent as null, so a sparse multi station feed stays small and a missing key reads naturally as "no reading". Choose the one file per channel layout and each message narrows to `{"timestamp": ..., "value": ...}` on its own topic, which is usually the easiest shape to bind straight onto a tag.

WAY OUT THREE

The read API, pulled on your schedule

You connect to us, over HTTPS on TCP 443

For a warehouse, a BI refresh, a GIS layer or a consultant's model, where you want the data on your own timetable and would rather not have anything written into your network. Read only by design, with one exception a key has to be granted: reporting what a crew found on a work order FlowSense raised. A key without that grant has no write path at all, so if it leaks it cannot change a single value.

What you provide	Nothing. An administrator issues an API key in the platform and gives it to you. It is displayed once and stored only as a hash.
What it can see	Exactly what the person who created it can see, optionally narrowed to named sites. Channels an administrator has hidden are invisible here too. If that person's access is reduced, the key follows, so offboarding somebody closes their integrations with them.
Endpoints	Under `/api/v1/public/v1` : `/whoami` , `/sites` , `/sites/{id}/channels` , `/readings/latest` , `/readings` and `/alarms` . Timestamps are ISO 8601 UTC. A key granted the work order permission may also `POST /work-orders/update` , the one write.
Limits	120 requests a minute per key. A readings request returns up to 5,000 points by default and 50,000 at most, over a window of up to 92 days; when the answer says it was truncated, ask again from the last timestamp you received. For a bulk historical extract, ask for a file feed instead. It is built for volume and will be faster than paging.
When it fails	Over the rate limit returns 429 with a Retry-After header. Unknown, revoked, expired and disabled keys all return the same 401, so the response cannot be used to work out which keys exist.

ONE REQUEST AND THE ANSWER TO IT

```
GET /api/v1/public/v1/readings?channel_id=1841&start=2026-09-16T00:00:00Z&end=2026-09-16T06:00:00Z
Authorization: Bearer eds_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

{
  "channel_id": 1841, "site_id": 204, "name": "Flow", "units": "L/s",
  "start": "2026-09-16T00:00:00Z", "end": "2026-09-16T06:00:00Z",
  "count": 72, "truncated": false,
  "readings": [
    {"timestamp": "2026-09-16T02:05:00Z", "value": 11.98},
    {"timestamp": "2026-09-16T02:10:00Z", "value": 12.05}
  ]
}
```

WAY OUT FOUR

Signed webhooks, the moment something happens

We post to an endpoint you publish, over HTTPS

Events rather than telemetry. If the point of the integration is to page somebody, raise a work order or light up a control room banner, polling for it adds latency you do not need. Four events are published: an alarm rule starting to breach, the same rule clearing, forecast rainfall putting a site at risk, and Blockage Watch flagging a developing blockage or raising its severity.

What you provide	A public HTTPS endpoint that answers with any 2xx inside 10 seconds. Plain HTTP is refused, and so are loopback and private addresses, so a webhook cannot be pointed back inside our own network.
Proving it came from us	Every request carries `X-EDS-Signature: t=...,v1=...`, an HMAC-SHA256 over the timestamp and the raw body, keyed with a secret shown once. The timestamp is inside the signed material, so a captured request cannot be replayed later.
Deduplicating	Each delivery carries an `event_key` that is stable for that occurrence. A retry after a network failure can deliver the same event twice, so key your handler on it.
When it fails	Six attempts, spaced 1, 5, 15, 30 and 60 minutes, then abandoned. An endpoint that has failed for 24 hours and twenty attempts is switched off until somebody re-enables it, rather than posted to for ever. Deliveries are queued by a background job rather than sent inline, so a slow endpoint of yours never delays anything else, and every attempt is visible in the delivery log.

AN ALARM AS IT ARRIVES

```
POST /your/endpoint
X-EDS-Signature: t=1758002043,v1=<hex hmac-sha256>

{
  "id": 1234,
  "event": "alarm.triggered",
  "event_key": "alarm_event:412:triggered",
  "created_at": "2026-09-16T02:14:03.221Z",
  "data": {
    "event_id": 412, "site_id": 204, "site_name": "Milton St PS",
    "rule_id": 9, "rule_name": "High level", "rule_type": "threshold",
    "operator": ">", "threshold": 1.8, "value": 2.04, "condition_finding_id": null,
    "triggered_at": "2026-09-16T02:13:58Z", "cleared_at": null
  }
}
```

And without any integration at all

Not every request for data is worth a project. Any chart on screen exports to CSV, and a report can be scheduled to arrive by email daily, weekly or monthly, as a branded PDF, a CSV, or both. The interactive export is aligned to a one row per minute

grid so that channels logging at different intervals line up in a spreadsheet; the feeds described above carry readings at their own cadence, untouched. If somebody needs a number rather than a connection, send them the report.

4. THE FILE, COLUMN BY COLUMN

Written to your format, not ours.

A loader that has to be persuaded to accept a file is a loader somebody will eventually stop maintaining. Everything below is set per feed, so the file arrives in the shape your side already reads, and the same settings govern an MQTT payload.

Time zone	Any zone by name, such as `Australia/Brisbane`, or a fixed offset. By default the file opens with `# Timestamp timezone: ...` above the header, so the file states its own zone instead of relying on a convention nobody wrote down. That line can be turned off for a strict parser.
Timestamp column	Always first, headed `timestamp`, formatted with a pattern you choose. The default is `2026-09-16 02:05:00`; ISO 8601, date and time in one field or a bare epoch are all a pattern away.
Column headers	The channel's display name with its units, prefixed by the station name when the feed spans more than one site. A per channel alias overrides both, so the header can be your own tag or field name exactly as your loader expects it. Aliases are worth setting on any long lived feed: they pin the header text, so renaming a channel in the platform cannot move a column under your importer.
Values	As stored. Rounded to a fixed number of decimal places if you want them that way, and otherwise sent at full precision. A timestamp where a channel has no reading is left blank.
Layout	One consolidated file with a column per channel, one file per station, or one narrow `timestamp,value` file per channel. The same choice groups MQTT messages, so a feed can be switched between transports without your side seeing different data.
Delimiter and quoting	Comma, semicolon, tab or pipe. Quoting follows RFC 4180, so a name containing the delimiter cannot shift your columns.
File names	A template built from `{feed}`, `{site}`, `{channel}`, `{date}`, `{datetime}`, `{time}` and `{seq}`. Anything your watcher matches on can be encoded in it.
Encoding	UTF-8 with CRLF line endings, header optional, and empty windows skipped rather than sent as a header with no rows.
What can be in it	Any channel the feed's owner can see: measured flow, level, depth, velocity, rainfall, battery and signal, alongside derived channels such as computed flow from a depth and velocity pair, and offset corrected channels. Channels from any site and any logger type can sit in one feed.

5. CADENCE, WINDOWS AND WHY THERE ARE NO GAPS

The feed knows what it has already sent.

A feed is not a scheduled export that dumps the last 24 hours and hopes. It keeps a watermark: the timestamp of the newest reading successfully delivered. Every run exports what is newer than that, and the watermark moves only when the whole run has landed.

SETTING	RANGE	WHAT IT IS FOR
Interval	1 minute to 31 days	How often the feed runs. Five to fifteen minutes suits a control room; nightly suits an ETL.
Send at	A local clock time	Anchors a daily or longer feed to a time of day, so the file is on your server before the job that reads it starts.
Overlap	0 minutes to 7 days	Re-reads a little behind the watermark on every run, so readings that arrive late, which is normal for loggers reporting twice a day, are still forwarded rather than stranded behind it.

SETTING	RANGE	WHAT IT IS FOR
Backfill on first run	0 to 90 days	How much history the first delivery carries, so your system starts with a usable record rather than from the moment the feed was switched on.
Retry	5, 10, 20, 40, 60 minutes	Applied after consecutive failures and never slower than the feed's own interval. No run overlaps another on the same feed.
Empty windows	Skipped or sent	A quiet period sends nothing by default, so an empty file never reaches your loader.

The trade this makes, stated plainly

If one file of three fails to upload, the whole window is re-sent on the next run. Your server may then receive a file it has already seen. That is deliberate: a duplicate file is recoverable at your end, a lost reading is not, and the alternative design silently drops data when a transfer half completes. Deduplicate on station and timestamp, which every historian and SCADA importer already does, and the trade costs you nothing.

Every run is recorded with its window, row count, file count, byte count and the remote paths it wrote, successes and failures alike, and a run interrupted by a restart is marked failed rather than left looking as though it is still going. When somebody asks whether last Tuesday's file was sent, the answer is on the screen with the path it was written to.

6. PORTS AND DIRECTION

What your firewall actually has to allow.

Pulling from us needs no rule at all. A file or MQTT feed needs one service you already run to be reachable from a single address, which we give you in writing, and a webhook needs an endpoint most organisations already publish.

WAY OUT	DIRECTION	PORT	WHAT YOU ALLOW
Scheduled files, SFTP	EDS to you	TCP 22, or your port	Your SFTP service reachable from a single EDS source address, and an account confined to one folder.
Scheduled files, FTPS	EDS to you	TCP 21 or your port, with explicit TLS, plus your passive range	The same, with TLS. We verify your certificate rather than accepting whatever answers.
MQTT publish	EDS to you	TCP 8883 with TLS, or 1883	Your broker reachable from that address, and a publish only account on the topic prefix.
Read API	You to EDS	TCP 443 outbound	Nothing inbound. Standard outbound web access from whatever runs the pull.
Webhooks	EDS to you	TCP 443	One public HTTPS endpoint. Nothing else, and nothing on your control network.
Browser and email	You to EDS	TCP 443, plus mail	Outbound web access for staff, and the sending domains allowlisted if scheduled reports are used.

7. SECURITY AND CONTROL

Outbound data is a permission, not a default.

In transit	SSH for SFTP, TLS for FTPS, MQTT and HTTPS. Host name checking is never switched off, including where a pinned internal CA or self signed certificate is in use.
Proving it is your server	Encryption alone only stops a passive eavesdropper: anyone who can answer in your server's place still collects the credential. Your SSH host key or certificate is therefore pinned at setup, shown to the operator before it is trusted, and confirmed with you out of band. For SFTP the check runs before authentication, so a mismatch aborts before the password is offered.
Your credentials with us	Encrypted at rest with authenticated encryption and never returned by any interface after entry, to you or to us. A password or key can be replaced, never read back.

Least privilege at your end	A folder confined file account, or a publish only broker account, is all a feed needs. We never ask for shell access, an administrative account, or write access anywhere but the one folder or topic.
Who can create one	Access to Data Export, where feeds are built, is granted per person and off by default, unlike every other page permission, because it sends telemetry to a server outside the platform. Only a platform administrator can grant it, and a feed is visible only to the account that owns it.
Keys and tokens	API keys are read only, apart from the one work order write a key must be granted, shown once, stored as a hash, revocable instantly, and bounded by their creator's own access. Webhook payloads are signed, and the signing secret is shown once.
Audit	Every delivery, every webhook attempt and every administrative change to a feed is recorded, with the window, the destination and the outcome.
Fuller answers	The **technical and security brief** answers a security questionnaire in full, including access control, encryption, logging and data handling. The **integration guide** is the developer reference for the read API and signed webhooks, with working verification code.

8. COMMISSIONING

From decision to standing feed, usually in an afternoon.

01 **Decide where the data has to land**

A folder, a broker topic, a script that pulls, or an endpoint that listens. Section 2 is the shortcut: pick what your team already supports.

02 **You create the destination**

A folder confined account, a publish only broker account, or nothing at all if you are pulling. Send us the host, port, folder or topic prefix, and the fingerprint of the host key or certificate, by a route other than the one we are about to connect over.

03 **We build the feed and pin your server**

Channels selected, format set, peer scanned and pinned. The connection test writes and removes a probe file, or publishes to a test topic, so the credential and the path are proven before any data moves.

04 **You approve the exact file**

A preview builds the real file from real readings and shows it to you before anything is sent. Column names, time zone, delimiter and precision are settled here, on a sample your loader can be run against, rather than after three nights of rejected files.

05 **First run, watched**

The feed is run on demand and the run log checked: rows, files, bytes and the remote paths written. Your side confirms the file was picked up and parsed.

06 **It becomes a standing feed**

From then it runs on its schedule with nobody attending it. Adding a station later is a channel added to the feed, not a new integration.

WHAT WE NEED FROM YOUR IT OR SCADA TEAM

- ☐ A **destination account** confined to one folder or one topic prefix, with a password or an SSH public key we supply.
- ☐ The **host name, port and fingerprint** of the host key or TLS certificate, sent out of band.
- ☐ Our **source address allowlisted**, if the service is filtered. We send it in writing.
- ☐ A decision on **time zone, delimiter, decimal places and file name pattern**, or a note to use the defaults.

- [ ] The **tag or field names** your system expects, if the columns should carry your names rather than ours.
- [ ] For webhooks, a **public HTTPS endpoint** that answers 2xx within 10 seconds, and somewhere to keep the signing secret.

9. WHAT USUALLY GOES WRONG

Nine things, and what each one actually is.

These are the questions that come back in the first fortnight of a file based integration, ours and everybody else's. Most have a setting rather than an answer.

WHAT YOU SEE	WHAT IT IS
The importer occasionally reads a truncated file	A folder watcher reading a file still being written. Atomic upload prevents it and is on by default; check it has not been switched off for a server that cannot rename.
The same file arrived twice	Expected after a failed or partial run. The window is re-sent rather than skipped. Deduplicate on station and timestamp.
Everything is ten hours out	The feed's time zone against your loader's assumption. The file names its zone in the comment line above the header; set the feed to whatever your loader expects, or switch it to UTC.
A column moved and the loader mapped the wrong tag	A channel was renamed in the platform, and the header follows the display name. Set a column alias per channel to pin the header text permanently.
Two files overwrote each other	Two stations with the same name on a per station feed, since the file name is built from it. Station names are not unique across councils; add `{seq}` or `{datetime}` to the template, or use aliases.
Readings appear in our system a day late	Loggers that report in batches twice a day. The reading carries its true field time, not its arrival time, so the row is correctly dated but arrives later. Set an overlap window so late arrivals are still forwarded.
MQTT messages are missing and nothing is logged	QoS 0, where the broker never acknowledges and a dropped publish cannot be detected. Use QoS 1, which is the default.
The API started returning 429	Over 120 requests a minute. Page through with the truncation flag rather than parallelising, and move a bulk extract to a file feed.
The run log says skipped and no file arrived	An empty window, with empty deliveries skipped. Nothing is wrong: no readings were recorded in that period for the channels selected.

10. QUICK ANSWERS

The questions that come back from a review, answered in a sentence each.

Does anything have to be installed on our network?	No. A destination account on a server you already run, or nothing at all if you pull from us.
Do you need access into our control network?	No. We write to one folder or publish to one topic, and never to a plant protocol. OPC UA, DNP3 and Modbus are not touched in either direction.
Is the data ours?	Yes. It is your telemetry, and the feeds exist so that holding it in FlowSense never becomes a reason you cannot use it elsewhere.
Can we take everything with us if we leave?	Yes. A file feed can carry every channel you hold, the read API covers the same ground, and the engineering tools export as SWMM, ICM, GeoJSON and Excel. Open formats, no export fee.
Can a feed carry our history as well as new readings?	Up to 90 days of it on the first run, which is enough for most systems to start with a usable record. Anything longer is supplied as a one-off extract rather than through the feed.

Do you change the readings before sending them?	No. Values leave as they are stored, and a feed never fills, smooths or conditions them on the way out. A stored value changes only through a repair a person applies, and the value it replaced is recorded.
What happens to a file if our server is down?	Nothing is lost. The window stays open, the retries back off, and the whole window is delivered when your server answers again.
Can a leaked API key change our data?	No. Nothing in the public API writes a reading. Its one write, a crew's finding on a work order, works only for a key granted it, so the worst case is a key that reads what its creator could read, and it is revoked instantly.
How do we know a webhook is really from you?	Verify the HMAC-SHA256 signature over the raw body, with the timestamp inside the signed material so a captured request cannot be replayed.
Can different teams get different data?	Yes. A feed carries only the channels selected for it, and an API key never sees more than the person who created it.
Does a feed slow anything down at our end?	No. A file lands in a folder and a message on a topic. Nothing polls your systems, nothing holds a session open, and a feed that fails retries on its own rather than calling anybody.
How many feeds can we have?	As many as you need. A nightly file for the historian, a five minute MQTT feed for the control room and a weekly extract for a consultant can all run at once from the same record.
What does it cost?	Nothing extra. Forwarding, the read API and webhooks are part of the platform, priced per monitored site, not per integration.
Who do we talk to?	1300 721 683, and ask for the engineer looking after your organisation, or reply to the email this arrived with.

Travels with this	The **SCADA connection brochure** covers the five ways in, if readings from your own SCADA should also appear in FlowSense. The **technical and security brief** answers a security questionnaire. The **integration guide** is the developer reference for the read API and webhooks.
Worth deciding early	Which system is the system of record for a given station. A feed makes it easy for the same readings to live in three places, and the time to say which one an operator should trust is before the third one exists.

NEXT STEP

Tell us what has to receive the data, and we will build the feed.

An EDS engineer will take your control systems or IT team through the options on a call, say which is least work for what you already run, send the source address for any firewall change, and confirm your server's fingerprint out of band before a credential is used. You will see the exact file, built from real readings, before a single one is sent.

Environmental Data Services Australian owned and operated since 1991
Head office 13/20-22 Ellerslie Road, Meadowbrook QLD 4131
Offices in Brisbane, Richmond NSW, Boronia VIC and Adelaide SA

1300 721 683 eds@e-d-s.com.au
www.e-d-s.com.au · edsflowsense.au

EDS FlowSense outbound data brochure, October 2026. Every format, limit and behaviour described is read from the released platform at the date of issue. Station names in the samples are illustrative.

DOCUMENT CONTROL

DOCUMENT	REFERENCE	REVISION
EDS FlowSense outbound data brochure	EDS-FS-OUT-01	**1.3**, issued 11 October 2026
SUPERSEDES	**STATUS**	**ISSUED BY**
Revision 1.2, 9 October 2026	Issued for client use	Environmental Data Services, ABN 31 729 813 151